





when (x) {  }

when (x) {



x.balanced += 10;





An Axiomatic Concurrency Model

4

2

S



R



C



Each behaviour has three lifecycle events

Executions are tuples of (E, po, co)



E are the events of the program

S



po



po

R



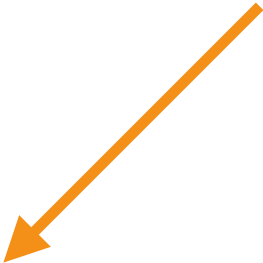
C





po

po is the program order



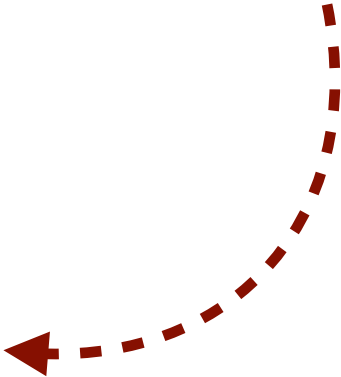
co is the cow n order

COX





is the derived run order



h

b

$h\dot{h}$ is the derivative of h happens before order

if behaviours share a common and have a path between their spans then complete happens-before run





when (x) {  **}**

when (x) {



x.balance += 10;

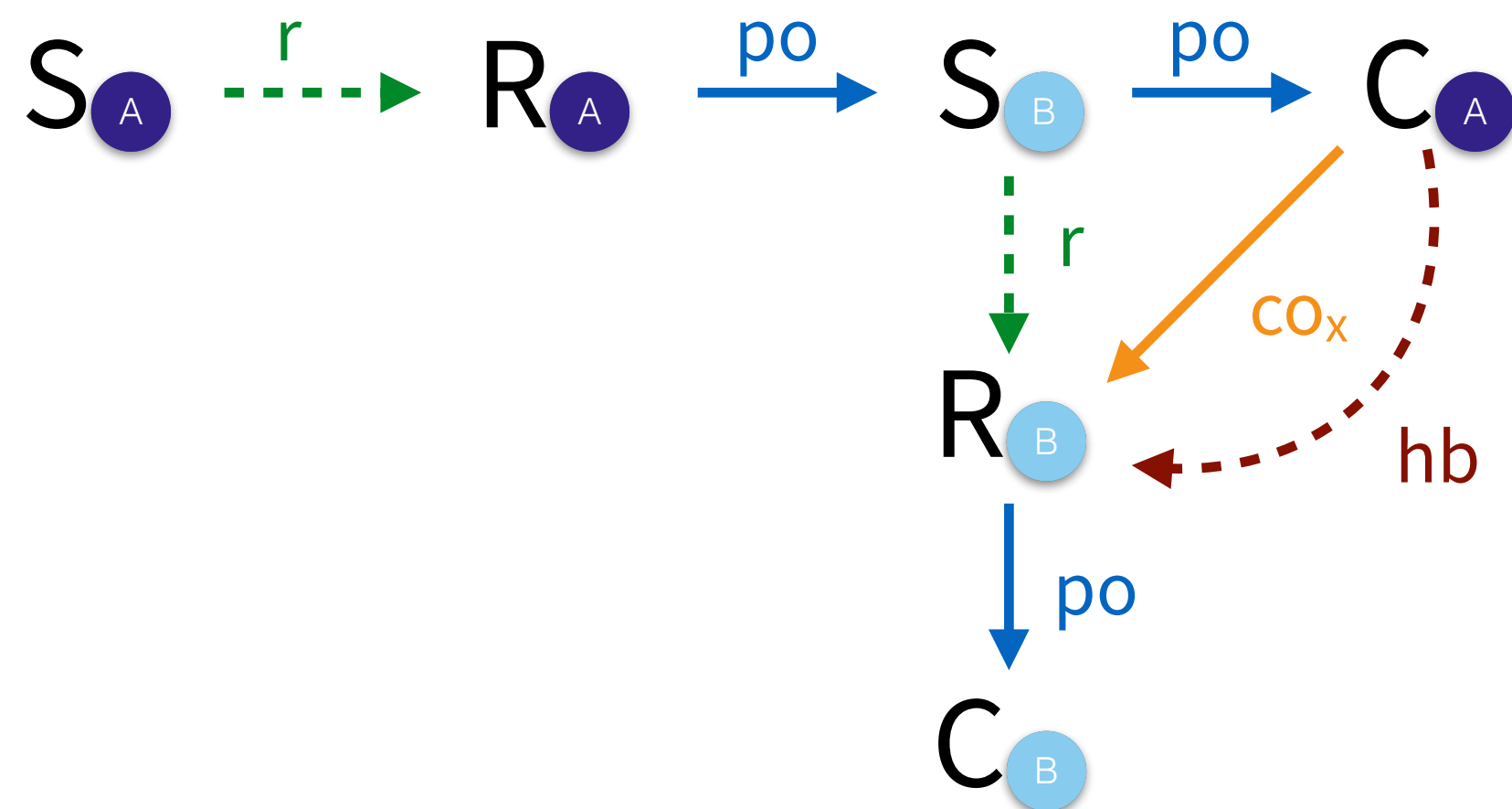
valid ifacyclic

An Axiomatic Concurrency Model

```
when(x) { A
  x.balance += 10;
  when(x) { B }
}
```

Executions are tuples of $(\mathbb{E}, po, co)$

Each behaviour has three lifecycle events



$\mathbb{E}$ are the events of the program

po is the program order

co is the cown order

r is the derived run order

hb is the derived happens before order

Valid if acyclic

If behaviours share a cown and have a path between their spawns then complete happens-before run

Validating Causality

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
    } } }
```

update(src, +100)

update(src, -100)